

PROYECTO BASES DE DATOS

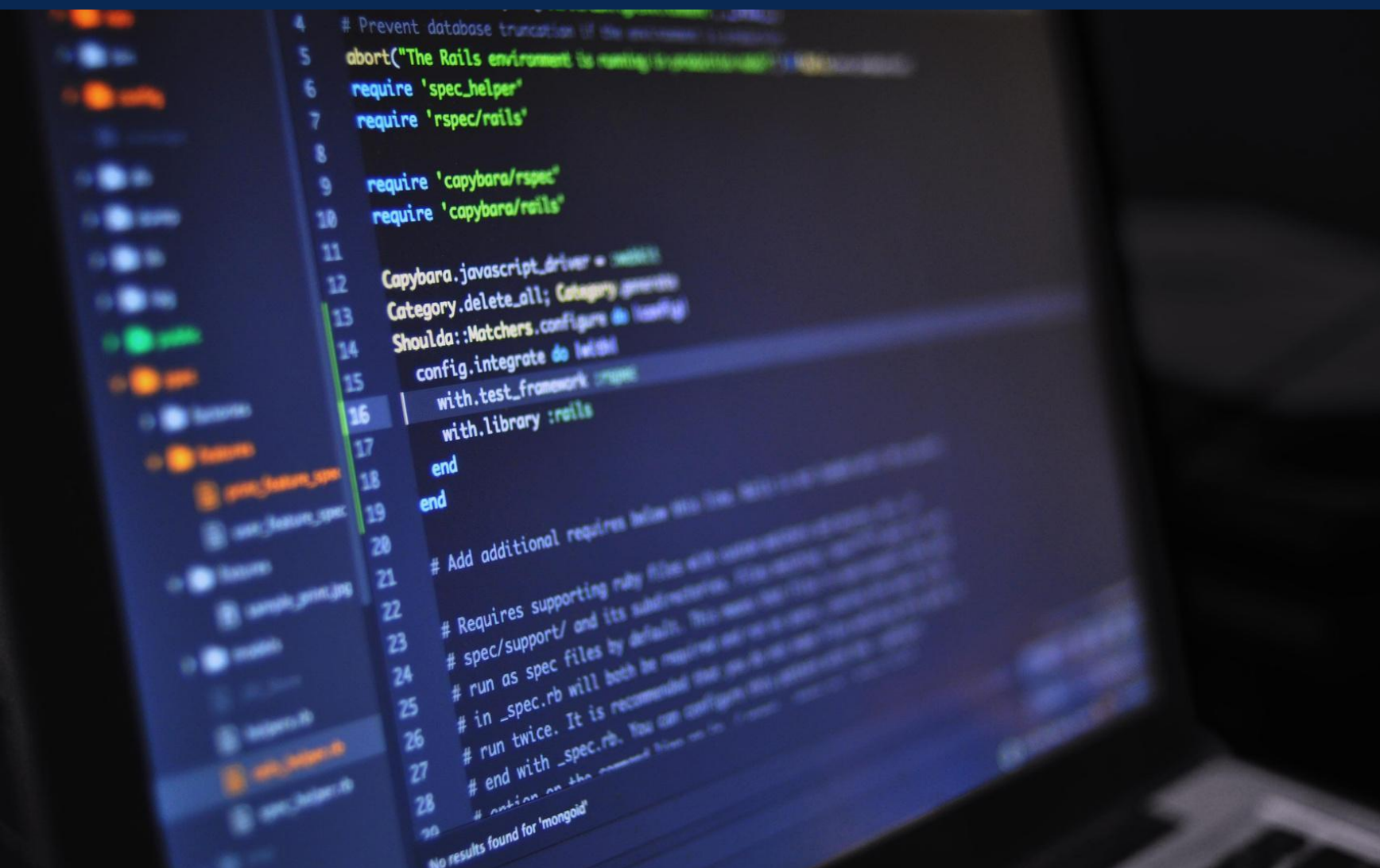
MONGO DB

REDIS

NEO4J

INDICE DE CONTENIDOS

- QUÉ DATOS GUARDAMOS EN CADA BASE DE DATOS
- PORQUE ELEGIMOS CADA BASE DE DATOS PARA ALMACENAR ESOS DATOS
- CÓMO CARGAMOS TODOS LOS DATOS
- ALGUNAS MUESTRAS DE RESULTADOS



**QUE DATOS GUARDAMOS EN CADA
BASE DE DATOS**

MONGO DB

- Información de usuarios
- Destinos turísticos (incluyendo actividades)
- Información de hoteles
- Historial de reservas confirmadas.



REDIS

- Caché de búsquedas
- Usuarios conectados
- Reservas temporales (aún no concretadas)

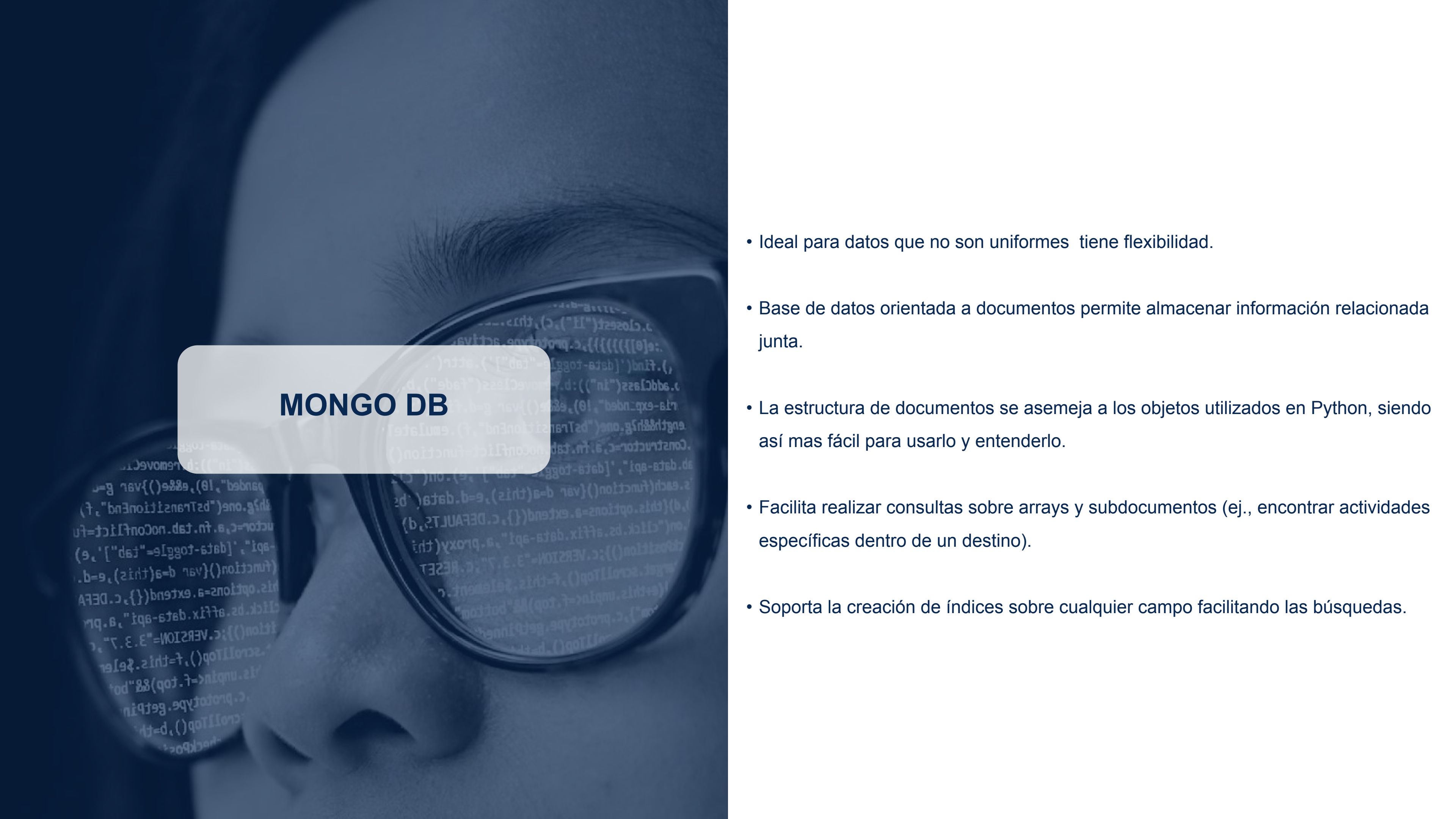


NEO4J

- Relaciones entre usuarios y destinos
- Relaciones entre usuarios



**PORQUE ELEGIMOS CADA BASE DE
DATOS PARA ALMACENAR ESTOS
DATOS**



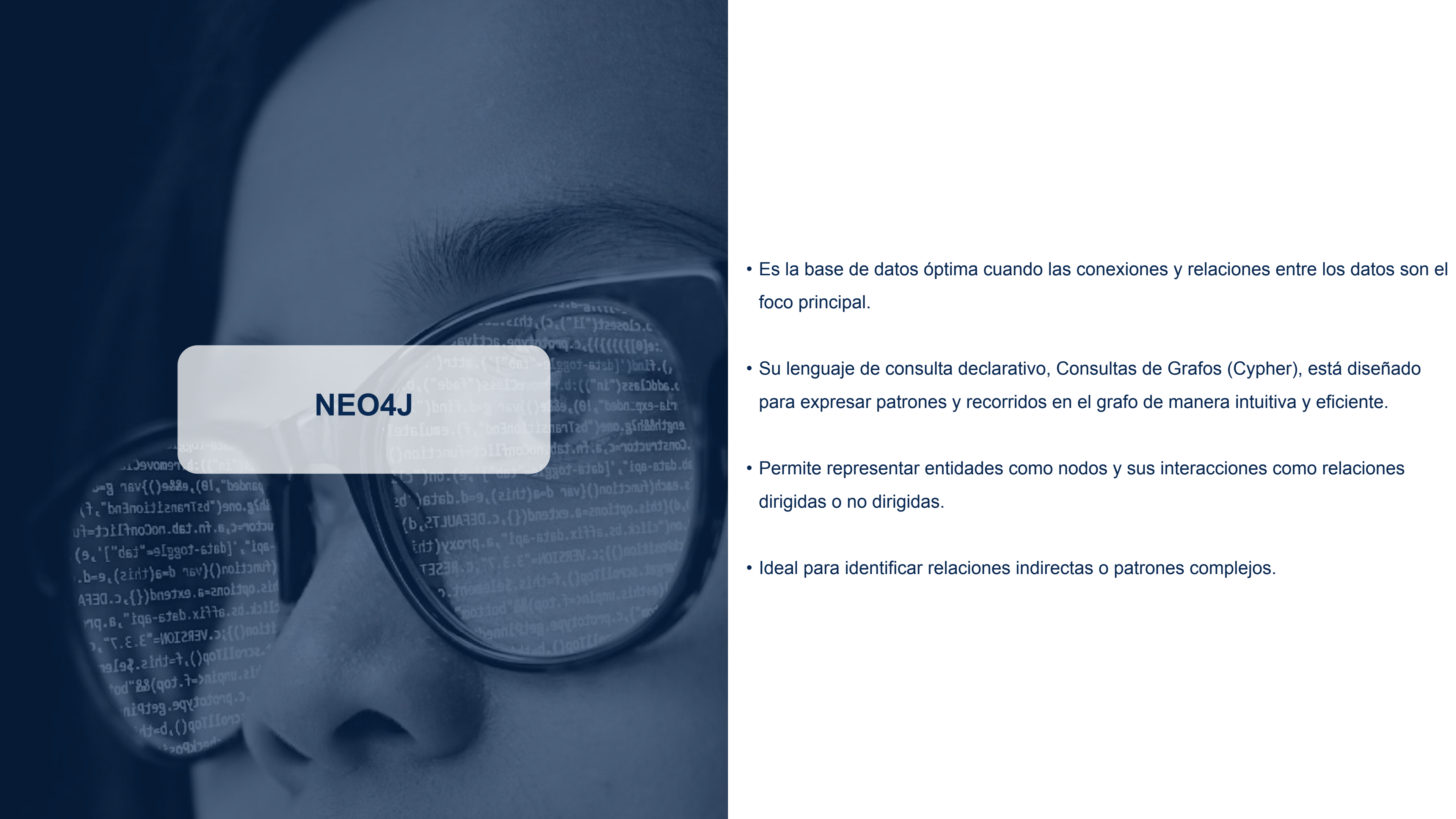
MONGO DB

- Ideal para datos que no son uniformes tiene flexibilidad.
- Base de datos orientada a documentos permite almacenar información relacionada junta.
- La estructura de documentos se asemeja a los objetos utilizados en Python, siendo así mas fácil para usarlo y entenderlo.
- Facilita realizar consultas sobre arrays y subdocumentos (ej., encontrar actividades específicas dentro de un destino).
- Soporta la creación de índices sobre cualquier campo facilitando las búsquedas.



REDIS

- Al operar en memoria RAM tiene latencias bajas para lecturas y escrituras.
- Proporciona tipos de datos como Sets, Lists y Hashes.
- Permite definir un tiempo de vida para las claves (expire), ideal para datos temporales que deben limpiarse automáticamente.
- Es utilizado como caché para almacenar datos accedidos frecuentemente.
- Con pipelines permiten agrupar múltiples acciones en una sola solicitud, mejorando la eficiencia de operaciones.



NEO4J

- Es la base de datos óptima cuando las conexiones y relaciones entre los datos son el foco principal.
- Su lenguaje de consulta declarativo, Consultas de Grafos (Cypher), está diseñado para expresar patrones y recorridos en el grafo de manera intuitiva y eficiente.
- Permite representar entidades como nodos y sus interacciones como relaciones dirigidas o no dirigidas.
- Ideal para identificar relaciones indirectas o patrones complejos.

CÓMO CARGAMOS TODOS LOS DATOS

```
# Datos base para la carga
names = ["Juan", "María", "Carlos", "Ana", "Pedro", "Sofía", "Pablo", "Laura", "Diego", "Valeria", "Gael", "Emilia", "Facundo", "Catalina", "Lucas",
"Julieta", "Martín", "Valentina", "Mateo", "Camila"]

last_names = ["Gómez", "Rodríguez", "Fernández", "López", "Martínez", "Díaz", "Pérez", "González", "Sánchez", "Romero", "Alvarez", "Ruiz", "Torres", "Flores",
"Benítez"]

destination_cities = ["Buenos Aires", "Córdoba", "Jujuy", "Rosario", "Mendoza", "Tucumán", "La Plata", "Mar del Plata", "Salta", "Santa Fe", "Corrientes",
"Neuquén", "Bahía Blanca","Bariloche", "Saladillo", "Ushuaia"]

hotel_suffixes = ["Resort", "Suites", "Inn", "Boutique", "Palace", "Plaza", "Grand Hotel", "Hotel"]

types_activities = ["Aventura", "Gastronómico", "Cultural", "Rural"]

activities = {
    "Aventura": ["Senderismo", "Canotaje", "Tirolesa", "Rappel", "Mountain Bike"],
    "Gastronómico": ["Ruta de vinos/cervezas", "Clase de cocina local", "Degustación de quesos", "Mercado de alimentos"],
    "Cultural": ["Visita a museos", "Recorrido histórico", "Teatro/Espectáculo", "Galería de arte"],
    "Rural": ["Día de campo en estancia", "Cabalgata", "Visita a granjas", "Recolección de frutos"]
}
```

"SIEMPRE CON COHERENCIA EN LOS DATOS"



ALGUNAS MUESTRAS DE RESULTADOS

```

# Punto 2.a.
print("\n--- Usuarios que visitaron 'Bariloche' ---")
if db is not None:
    # 1. Encontrar todas las reservas para "Bariloche" y obtener los IDs de usuario (sin repetir)
    user_ids_en_bariloche = db.reservas.distinct("user_id", {"destino_ciudad": "Bariloche"})

    if user_ids_en_bariloche:
        # 2. Buscar en la colección de usuarios aquellos cuyos IDs están en la lista que encontramos
        usuarios_encontrados = db.usuarios.find(
            {"id": {"$in": user_ids_en_bariloche}},
            {'_id': 0, 'id': 1, 'name': 1, 'last_name': 1} # Proyección para mostrar solo campos relevantes
        )

        print(f"Se encontraron {len(user_ids_en_bariloche)} usuarios que visitaron Bariloche:")
        for usuario in usuarios_encontrados:
            print(f"  - ID: {usuario['id']}, Nombre: {usuario['name']} {usuario['last_name']}")
    else:
        print("No se encontraron reservas para el destino 'Bariloche'.")

```

```

--- Usuarios que visitaron 'Bariloche' ---
Se encontraron 8 usuarios que visitaron Bariloche:
- ID: 4, Nombre: Julieta Pérez
- ID: 5, Nombre: Camila Rodríguez
- ID: 19, Nombre: Facundo Alvarez
- ID: 29, Nombre: Ana Martínez
- ID: 35, Nombre: Valentina González
- ID: 38, Nombre: Lucas Torres
- ID: 40, Nombre: Catalina Flores
- ID: 41, Nombre: Gael Gómez

```



```
# Punto 2.b.
print("\n--- Amigos de Juan y destinos compartidos ---")
query_b = """
MATCH (j:Usuario) WHERE j.nombre STARTS WITH 'Juan'
MATCH (j)-[:AMIGO_DE]-(f:Usuario)
MATCH (j)-[:VISITO]->(d:Destino)<-[:VISITO]-(f)
RETURN j.nombre AS nombre_de_juan, f.nombre AS nombre_del_amigo, d.city AS destino_compartido
"""

result_b = run_cypher(query_b)
if result_b:
    for record in result_b:
        print(f" - {record['nombre_de_juan']} es amigo de {record['nombre_del_amigo']} y visitaron {record['destino_compartido']}")
else:
    print("No se encontro a ningun Juan con destino compartido.")
```

```
--- Amigos de Juan y destinos compartidos ---
- Juan Flores es amigo de Valentina Rodríguez y visitaron Ushuaia
```

```

# Punto 2.c.
print ("Sugerir destinos para un usuario que no haya visitado él ni sus amigos")
nombre_result = run_cypher(f"MATCH (u:Usuario {{id: {USUARIO_OBJETIVO_ID}}}) RETURN u.nombre AS Nombre")
USUARIO_OBJETIVO_NOMBRE = nombre_result[0]['Nombre'] if nombre_result else f"ID {USUARIO_OBJETIVO_ID} (No encontrado)"

query_c = f"""
MATCH (u:Usuario {{id: {USUARIO_OBJETIVO_ID}}})
// Encuentra todos los destinos visitados por el usuario o sus amigos
MATCH (u)-[:VISITO|AMIGO_DE*2]->(d_visitado:Destino)
WITH collect(DISTINCT d_visitado.city) AS DestinosAExcluir

// Encuentra destinos que no estan en la lista de exclusión.
MATCH (d_sugerido:Destino)
WHERE NOT d_sugerido.city IN DestinosAExcluir
RETURN d_sugerido.city AS DestinoSugerido
"""

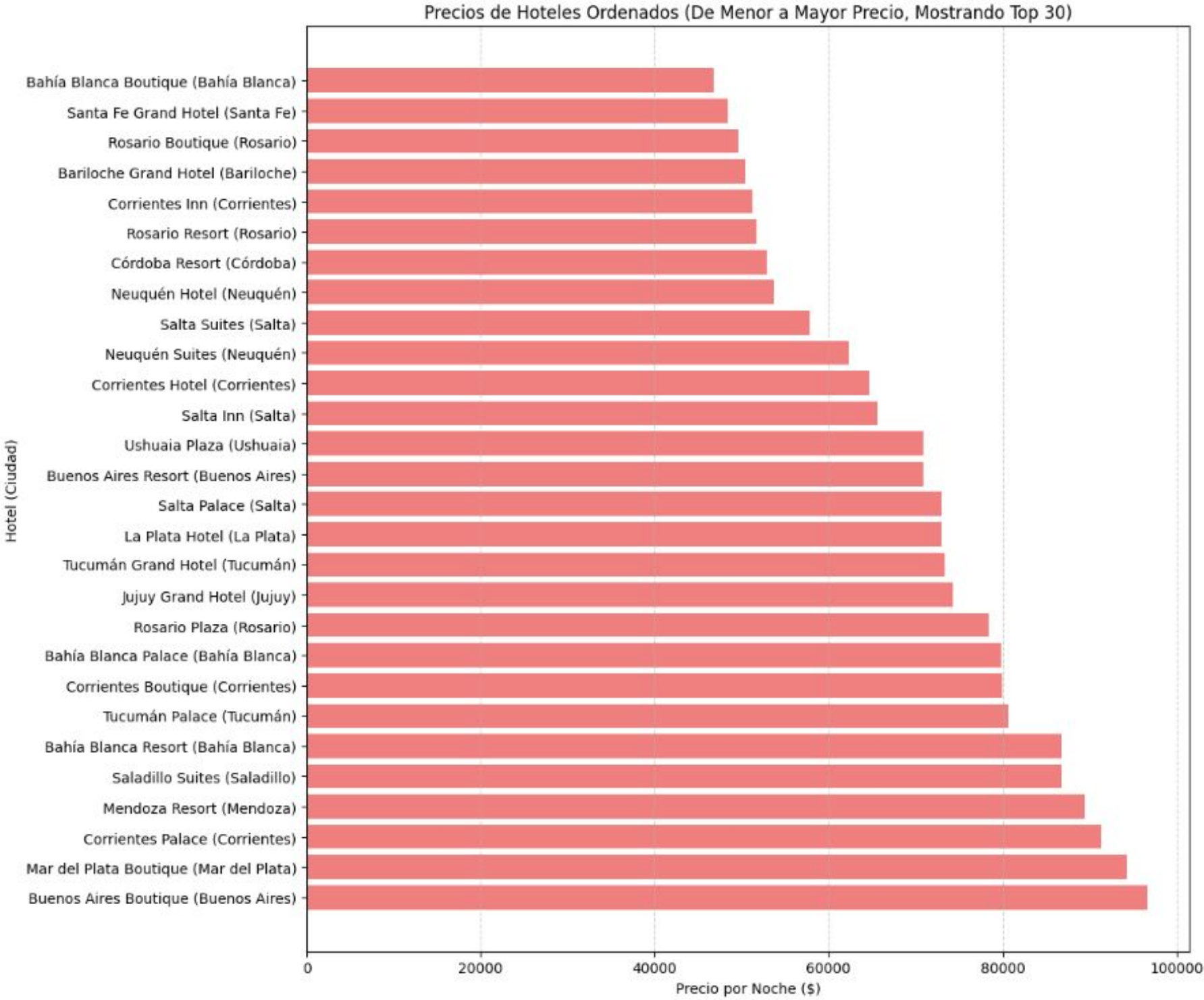
result_c = run_cypher(query_c)
print(f"Destinos sugeridos para {USUARIO_OBJETIVO_NOMBRE} con ID {USUARIO_OBJETIVO_ID}:")
if result_c:
    for record in result_c:
        print(f" - {record['DestinoSugerido']}")
else:
    print("No hay destinos para sugerir.")

```

Sugerir destinos para un usuario que no haya visitado él ni sus amigos

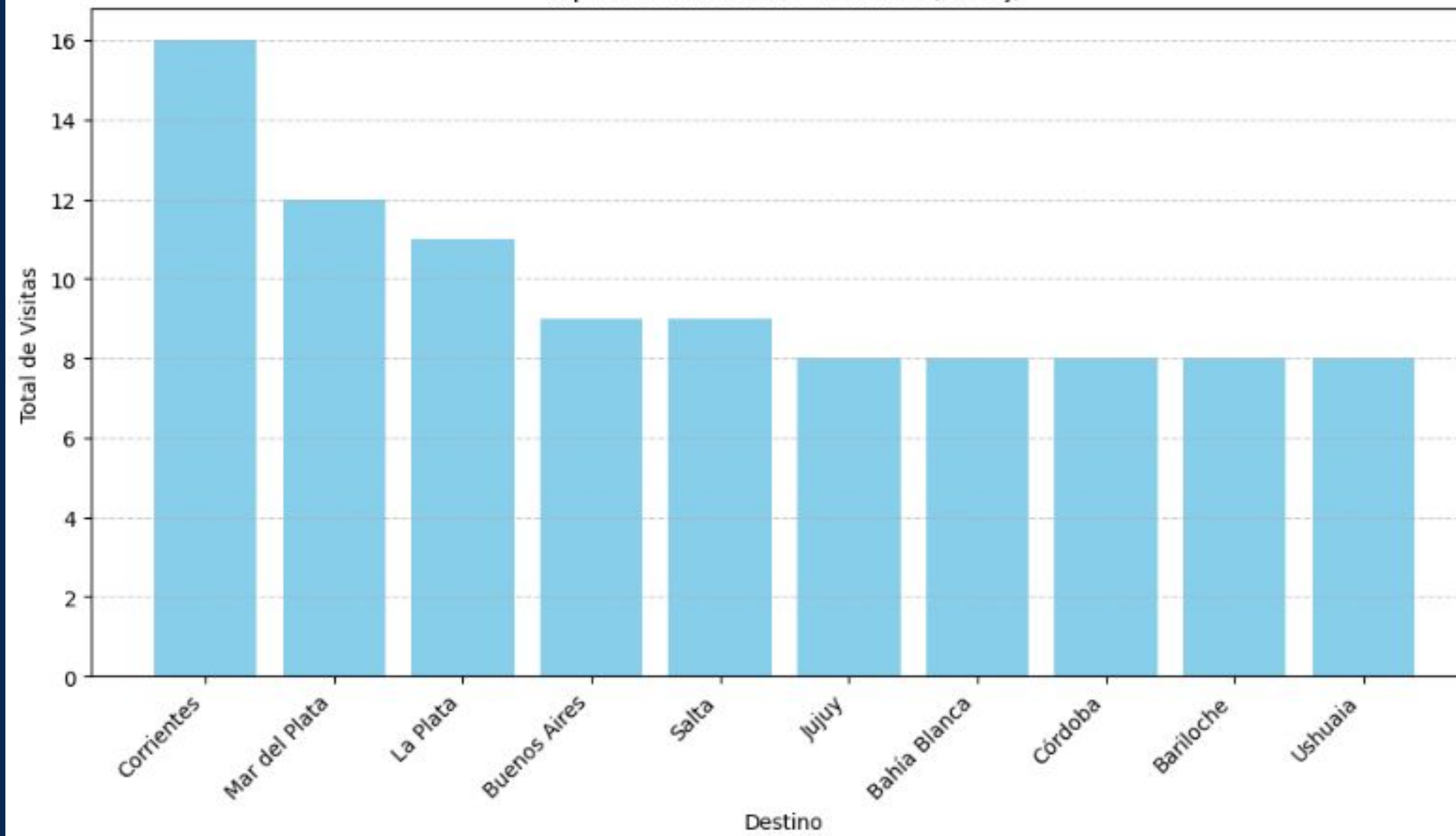
Destinos sugeridos para Laura Gómez con ID 21:

- Buenos Aires
- Córdoba
- Jujuy
- Rosario
- Mendoza
- Tucumán
- Mar del Plata
- Salta
- Santa Fe
- Corrientes
- Neuquén
- Bahía Blanca
- Bariloche
- Saladillo



--- Estadísticas ---
Número total de hoteles procesados: 121
Hotel más barato: Bahía Blanca Boutique en Bahía Blanca por \$46840.00.

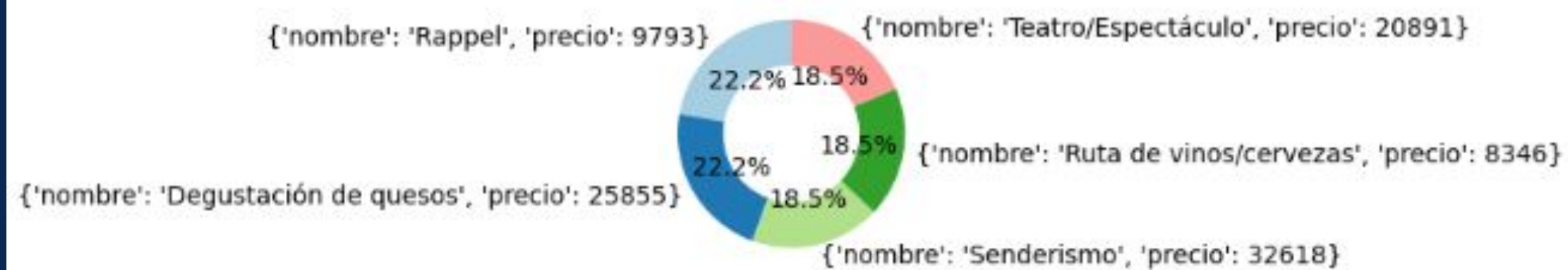
Top 10 Destinos Más Visitados (Neo4j)



Destino más visitado: Corrientes con 16 visitas.

Actividad mas Popular

Distribucion de Reservas de Actividades (Top 5)



Actividad más popular: {'nombre': 'Rappel', 'precio': 9793} con 6 reservas.

```

# Punto 2.g.
print("\n--- Usuarios conectados actualmente ---")
if r and db is not None:

    # 1. Obtenemos Los IDs de Los usuarios activos desde el Set de Redis
    usuarios_conectados_ids_str = r.smembers("usuarios_activos")

    if not usuarios_conectados_ids_str:
        print("No hay usuarios conectados en este momento.")
    else:
        # 2. Convertimos Los IDs (que Redis devuelve como strings) a números enteros
        user_ids_a_buscar = [int(uid) for uid in usuarios_conectados_ids_str]

        # 3. Buscamos en MongoDB TODOS Los usuarios necesarios en UNA SOLA CONSULTA
        usuarios_encontrados = db.usuarios.find(
            {"id": {"$in": user_ids_a_buscar}},
            {'_id': 0, 'id': 1, 'name': 1, 'last_name': 1} # Proyección para optimizar
        )

        print(f"✅ Se encontraron {len(user_ids_a_buscar)} usuarios conectados:")
        # 4. Iteramos sobre Los resultados de MongoDB y mostramos La información
        for usuario in usuarios_encontrados:
            print(f" - ID: {usuario['id']}, Nombre completo: {usuario['name']} {usuario['last_name']}")

else:
    print("❌ Para ejecutar esta consulta, se requiere una conexión activa tanto a Redis como a MongoDB.")

```

```

--- Usuarios conectados actualmente ---
✅ Se encontraron 18 usuarios conectados:
- ID: 1, Nombre completo: Diego López
- ID: 5, Nombre completo: Camila Rodríguez
- ID: 6, Nombre completo: Facundo Benítez
- ID: 11, Nombre completo: Diego Romero
- ID: 13, Nombre completo: Ana Torres
- ID: 15, Nombre completo: Sofía Gómez
- ID: 19, Nombre completo: Facundo Alvarez
- ID: 21, Nombre completo: Laura Gómez
- ID: 22, Nombre completo: Mateo Torres
- ID: 24, Nombre completo: Valentina López
- ID: 27, Nombre completo: Gael Sánchez
- ID: 28, Nombre completo: Sofía Gómez
- ID: 29, Nombre completo: Ana Martínez
- ID: 31, Nombre completo: Valentina Fernández
- ID: 34, Nombre completo: Pablo Fernández
- ID: 39, Nombre completo: Juan Gómez
- ID: 43, Nombre completo: Sofía Alvarez
- ID: 46, Nombre completo: Lucas Sánchez

```



```
# Punto 2.h.
print("\n--- Destinos con precio inferior a $100.000 ---\n")
if db is not None:
    # Operador '$lt' (Less than) para buscar precios menores a 100000
    destinos_baratos = db.destinos.find({"price": {"$lt": 100000}})

    resultados = list(destinos_baratos)
    if resultados:
        for destino in resultados:
            print(f" - {destino['city']}: ${destino['price']:,.}") # Formateamos el precio
    else:
        print("No se encontraron destinos con precio inferior a $100.000.")
```

--- Destinos con precio inferior a \$100.000 ---

- Bariloche: \$84,314

```
# Punto 2.i.  
print("\n--- Hoteles en 'Jujuy' ---")  
if db is not None:  
    destino_a_buscar = "Jujuy"  
    hoteles_encontrados = list(db.hoteles.find({"city": destino_a_buscar}))  
  
    if hoteles_encontrados:  
        print(f"Se encontraron {len(hoteles_encontrados)} hoteles en {destino_a_buscar}:")  
        for hotel in hoteles_encontrados:  
            print(f" - {hotel['name']} ({hotel['stars']} estrellas)")  
    else:  
        # Si no encontramos nada, damos una respuesta útil  
        print(f"No se encontraron hoteles para '{destino_a_buscar}'.")
```

```
--- Hoteles en 'Jujuy' ---  
Se encontraron 8 hoteles en Jujuy:  
- Jujuy Grand Hotel 1 (5 estrellas)  
- Jujuy Grand Hotel 2 (3 estrellas)  
- Jujuy Grand Hotel 3 (5 estrellas)  
- Jujuy Palace 4 (3 estrellas)  
- Jujuy Resort 5 (5 estrellas)  
- Jujuy Palace 6 (5 estrellas)  
- Jujuy Grand Hotel 7 (3 estrellas)  
- Jujuy Grand Hotel 8 (4 estrellas)
```

--- Incremento del 5% en actividades de Tucuman ---
Se actualizo el documento de Tucumán.

Tabla de comparacion de precios (Antes vs. Despues del 5%):

	Actividad	Precio_Original (\$)	Precio_Nuevo (\$)
0	Mountain Bike	24643	25875.15
1	Degustación de quesos	18462	19385.10
2	Clase de cocina local	20240	21252.00
3	Ruta de vinos/cervezas	14225	14936.25
4	Visita a museos	24692	25926.60
5	Teatro/Espectáculo	32475	34098.75
6	Recolección de frutos	14820	15561.00
7	Visita a granjas	21563	22641.15
8	Cabalgata	34191	35900.55


```

#Punto 3.b
print("\n--- Agregar servicio SPA al hotel ID=1 ---\n")

# Definicion del hotel y el servicio
hotel_id_a_modificar = 1
nuevo_servicio = "SPA"

# 1. Realizar la actualizacion
update_result = db.hoteles.update_one(
    {"hotel_id": hotel_id_a_modificar},
    {"$addToSet": {"servicios": nuevo_servicio}}
)

# 2. Imprimir los resultados
if update_result.matched_count > 0:
    # Consultar los servicios para mostrar el cambio
    servicios_actuales = db.hoteles.find_one({"hotel_id": hotel_id_a_modificar}, {"_id": 0, "servicios": 1, "name": 1})

    if update_result.modified_count > 0:
        print(f"- Se agrego el servicio '{nuevo_servicio}' al hotel {servicios_actuales.get('name')} (ID={hotel_id_a_modificar}).")
        print(f"  Nuevos servicios: {servicios_actuales.get('servicios')}")
    else:
        print(f"- El hotel {servicios_actuales.get('name')} (ID = {hotel_id_a_modificar}) ya tenia el servicio '{nuevo_servicio}'. No se realizaron cambios.")
        print(f"  Servicios actuales: {servicios_actuales.get('servicios')}")
else:
    print(f"❌ Error: No se encontro el hotel con hotel_id={hotel_id_a_modificar}.")

```

--- Agregar servicio SPA al hotel ID=1 ---

- Se agrego el servicio 'SPA' al hotel Buenos Aires Hotel 1 (ID=1).
 Nuevos servicios: ['Bar', 'Pileta', 'SPA']

Ingrese el ID del usuario que desea eliminar:

Ingrese el ID del usuario que desea eliminar: 4

--- Parte 1: Eliminando Usuario ID=4 de Neo4j ---

Usuario encontrado en Neo4j:

ID: 4

Nombre: Mateo Sánchez

Usuario 'Mateo Sánchez' (ID=4) y sus relaciones han sido eliminados de Neo4j.

--- Parte 2: Eliminando Usuario ID=4 de MongoDB ---

Usuario encontrado en MongoDB:

ID: 4

Nombre: Mateo Sánchez

Edad: 36

DNI: 65082282

Telefono: +5492738588842

Usuario (ID=4) eliminado de la coleccion 'usuarios' en MongoDB.

Se eliminaron 2 reservas asociadas al usuario ID=4 en MongoDB.

MUCHAS GRACIAS!

LOREDO LAUTARO, 18137/7
LUCENTINI JOAQUIN, 18143/6